

ZHIHONG (KATERINA) WANG

South Jordan, UT

801-520-0248 | work@katerina.wang | katerina.wang | [linkedin.com/in/katerinawang](https://www.linkedin.com/in/katerinawang)

Education

University of Utah, Kahlert School of Computing

Expected Dec 2027

Bachelor of Science in Computer Science, Games Emphasis | GPA: 4.0

Salt Lake City, UT

- Honors: Dean's List (Spring 2025, Fall 2025, Spring 2026, Summer 2026)

Relevant Coursework

Data Structures & Algorithms, Software Practice I & II, Computer Systems, Computer Organization, Web Development, Natural Language Processing (in progress), Data Analysis, Traditional & Alternative Game Development

Experience

Teaching Assistant (Lead TA, Fall 2026) | CS 3500 – Software Practice I

Aug 2025 – Present

University of Utah, Kahlert School of Computing

Salt Lake City, UT

- Serve as Lead TA, coordinating with the instructor and TA team on course operations and student communication.
- Conduct code reviews and one-on-one code discussions to assess students' program design, implementation decisions, and code structure.
- Grade 20+ submissions weekly with rubric-based feedback on code quality, design, and implementation.
- Lead lab sessions and office hours, troubleshooting code, Git/GitHub, and development-environment issues for 5+ students per session; answer student questions on Piazza.

Computer Science and Math Tutor

Jan 2026 – Present

STEM Learning Center, Salt Lake Community College

Salt Lake City, UT

- Tutor 15+ students per week in computer science and mathematics courses, including CS fundamentals, College Algebra, and Calculus I-II.
- Apply active-learning methods to build students' problem-solving skills and conceptual understanding.

Web Administrator

Aug 2016 – Dec 2019

Yunchuang Technology

Mianyang, China

- Built and deployed WordPress sites on Nginx; resolved production issues after an Apache-to-Nginx migration, including php-fpm permission mismatches, disabled PHP functions, and wp-config.php filesystem-method conflicts.
- Administered a centralized LNMP (Linux, Nginx, MySQL, PHP) server hosting 20+ domains; configured SSL/HTTPS with Let's Encrypt (Certbot) and automated cron renewal, blocked direct-IP access through Nginx virtual-host rules, and managed deployment and monitoring in BT Panel.
- Built local development environments with Vagrant, VirtualBox, and CentOS VMs, including multi-VM NAT and bridged networking; created a Travis CI pipeline to auto-deploy a Hexo blog to GitHub Pages.

Projects

TOG – Math Puzzle Game | GameMaker Studio 2, GML | GAMES 3720, AI-Assisted

May – Jul 2026

- Directed AI coding tools (OpenAI Codex) to build a grid-based math puzzle game on a 6-person team; wrote behavior specifications, ran testing and bug reproduction, and maintained the shared repository (47% of team commits).
- Specified and validated core mechanics: conditional walls with six comparison operators, an arithmetic tool row, an equation-history panel, ranked door progression, dialogue triggers, and graph auto-zoom with coordinate-grid snapping.
- Integrated AI-generated code by resolving merge conflicts in shared GameMaker project files, uncovering edge cases through playtesting (e.g., multi-block chain pushes through conditional walls), and iterating specs until behavior matched design intent.

Schrödinger's Curiosity | Unreal Engine 5, Blueprints, UMG, Perforce | GAMES 3710, Team Lead

May – Jul 2026

- Led development as Team Lead on a 6-person team, assigning tasks based on team members' strengths and managing communication across programming, art, and design.
- Built a time-reversal system that records and replays player controller state when the player switches between characters.
- Created a contextual UI system that displays key-binding prompts near interactive objects.
- Authored a UV-based material that plays 2D animations inside UMG widgets.
- Integrated assets and pipelines across disciplines, maintaining a consistent Unreal Engine project structure under Perforce version control.

GMTK Game Jam 2026 – Heart Beats per Minute | *Unity 6, C#, WebGL* | *Game Jam* **Jul 2026**

- Owned UI systems on a 5-person team and contributed camera and rhythm-gameplay code; built a UI controller managing five panel states, slide-out scene transitions, and player settings wired to game systems.
- Implemented the initial camera transition between the café conversation view and the falling-note rhythm view using lerp-based pose interpolation, AnimationCurve easing, and orthographic-to-perspective switching.
- Built core rhythm playback logic: a spawn-ahead window, configurable miss detection, note-lifecycle tracking with HashSets, and input-to-note hit matching; created the initial rhythm board and note foundations.
- Implemented pause (clock freeze and menu), settings persistence with PlayerPrefs (volume, note speed, difficulty), end-screen scoring, and WebGL browser autoplay-policy handling.

Catstronaut | *C#, .NET 9, Blazor, JavaScript, HTML5 Canvas* | *Hackathon, AI-Assisted* **Oct 2025**

- Won Best Game at Crimson Hackathon 2025** with a browser-based adventure game built in 5 hours by a 4-person team.
- Specified and directed AI generation of the canvas game engine: sprite animation, a requestAnimationFrame loop with delta-time physics, gravity/velocity/friction, and platform collision detection.
- Tested and debugged AI-generated code under deadline; delivered high-DPI canvas rendering (devicePixelRatio scaling), Blazor-to-JavaScript interop for SPA integration, and proximity-based level progression.

3D Platformer Level | *Unreal Engine 5, Maya, ZBrush, Substance 3D Painter* | *GAMES 3010* **May 2025**

- Designed and built a 3D platformer level using a full asset pipeline: modeling, sculpting, texturing, and engine integration.
- Modeled assets in Maya, sculpted detail in ZBrush, and textured assets in Substance 3D Painter.
- Assembled the level in Unreal Engine 5 with lighting, materials, and gameplay elements, including button-triggered moving and rotating platforms, checkpoints, and spike traps.

AI Trading Analysis System | *Python, MCP (FastMCP), Pydantic, pytest* | *Personal Project, AI-Assisted* **Sep 2026**

- Architected and directed AI (Claude) development of a two-part trading-analysis system: a Python MCP server exposing Robinhood market data via robin_stocks, and Claude skill files defining five specialist agents (orchestrator, market scanner, technical, options, and news analysts).
- Defined the server design with FastMCP, Pydantic data models, and typed error handling across 7 error codes; exposed tools for account data, OHLCV candles, options chains with Greeks, earnings, and market scans.
- Enforced safety by design with hard-stubbed execution tools, readiness tiers, and a written risk policy; directed a contract test suite with mocked robin_stocks plus 12 smoke tests tracked in a regression log.

Focus Tab – Chrome Extension | *JavaScript, Chrome Extensions API* | *Personal Project, AI-Assisted* **Sep 2026**

- Designed and directed AI development (Claude, ChatGPT) of a productivity new-tab extension spanning 11 vanilla JavaScript modules: draggable/resizable widget panels, Unsplash backgrounds with adaptive glassmorphism, a todo system with categories and subtasks, and a Markdown notes editor.
- Specified features as behavior requirements and owned testing and edge-case discovery; defined an SM-2 spaced-repetition tracker for LeetCode practice with bulk import, automatic schedule rebalancing, and a mastery threshold at review intervals of 30+ days.

From Autograd to Transformers | *Python, PyTorch, Hugging Face Transformers, Jupyter* | *Self-Study* **Jul 2026**

- Completed 10+ Jupyter notebooks across Karpathy's from-scratch series (micrograd, makemore) and a structured curriculum progressing from PyTorch fundamentals to Transformers and BERT fine-tuning.
- Built an autograd engine (computation graph, backpropagation via topological sort) and an MLP trained with manual SGD; verified all gradients against PyTorch autograd.
- Implemented scaled dot-product attention, multi-head attention, feed-forward layers, and a Transformer encoder block with residual connections and LayerNorm; stacked 3 blocks (149,952 parameters) and matched outputs against nn.TransformerEncoder.
- Fine-tuned BERT for sentiment classification with the Hugging Face Trainer API; ran ablations on frozen vs. full fine-tuning (1,538 vs. 109M trainable parameters), epoch count, and optimizer choice (Adam vs. SGD).

Custom Memory Allocator (malloc) | *C, x86-64, Linux* | *CS 4400* **Apr 2026**

- Built a dynamic memory allocator with 10 segregated free lists (16 B to 4096+ B), boundary-tag coalescing for all four neighbor cases, and block splitting to reduce fragmentation.
- Managed page-level heap growth via mem_map/mem_unmap with exponential expansion capped at 64 KB, returning fully freed pages to the OS.
- Guaranteed 16-byte alignment per the x86-64 ABI while balancing throughput and memory utilization.

Multi-Threaded HTTP Friend-List Server | *C, pthreads, Semaphores, Sockets* | *CS 4400* **Apr 2026**

- Built a concurrent HTTP server with a thread-per-connection model (`pthread_create/pthread_detach`) and binary-semaphore synchronization of shared data.
- Implemented HTTP endpoints for friend management (`/friends`, `/befriend`, `/unfriend`) and federated friend discovery (`/introduce`) via server-to-server requests.

Tiny Shell (tsh) | *C, POSIX Signals, Pipes, Linux* | *CS 4400*

Mar 2026

- Built a Unix shell supporting foreground/background jobs, built-in commands (`quit`, `jobs`, `bg`, `fg`), and pipelines via `pipe()` and `dup2()`.
- Wrote `SIGCHLD`, `SIGINT`, and `SIGTSTP` handlers, using `sigprocmask` to prevent race conditions between `fork()` and job-list updates.
- Used `sigsuspend()` for race-free foreground waiting and `tcsetpgrp()` to transfer terminal control to child process groups.

Cache Simulator & Optimizer | *Python, LaTeX* | *CS 3810*

Dec 2025

- Implemented a configurable cache simulator supporting direct-mapped, set-associative, and fully associative caches with LRU replacement and tag/index/offset address decomposition.
- Swept 280 configurations (block size, set count, associativity) over a 464 KB ray-tracing memory trace to identify optimal parameters; analyzed BVH-traversal spatial locality in a LaTeX report.

COAL Assembler | *Python, Regex, argparse* | *CS 3810*

Nov 2025

- Built a two-pass assembler for the 16-bit COAL ISA (13 instructions, 4-bit opcodes, 4 registers): pass one builds a label symbol table; pass two resolves references and emits binary machine code.
- Tokenized assembly with regex-based parsing, including LW/SW offset syntax, and reported errors for unknown instructions, undefined labels, and malformed operands.

Logic Gate Simulator | *C++, Qt 6, QGraphicsView* | *CS 3505*

Dec 2025

- Built an educational circuit-puzzle game in which players connect AND, OR, and NOT gates to solve eight progressively harder levels (5-person team).
- Designed Gate, Node, and Wire models with enum-based type dispatch and signal-propagation logic.
- Created a drag-and-drop gate palette with snap-to-slot placement and real-time wire signal visualization.
- Built level progression with sequential unlocking, timed challenges, and audio integration.
- Fixed destructor-ordering crashes, cross-platform drag-and-drop issues, and signal-propagation errors.

Sprite Editor | *C++, Qt 6* | *CS 3505*

Nov 2025

- Co-developed a 32x32 pixel-art editor with a frame timeline, bucket fill, and animation support as top contributor on a 6-person team (47% of commits).
- Built the 640x640 editing canvas with 20x zoom, variable brush size (1–32 px), and an eraser tool.
- Implemented bucket fill with a DFS flood-fill algorithm and migrated pixel storage from 2D arrays to QImage for native Qt rendering.
- Designed frame management (add, remove, copy, swap, clear) with a 128x128 thumbnail timeline.

DishBook – Recipe Platform | *Django, Python, JavaScript, SQLite, AWS* | *CS 3550*

Aug – Nov 2025

- Built a full-stack recipe-sharing platform across 7 iterative phases, modeling recipes, steps, and ingredients with cascading deletes, self-referential foreign keys for recipe variations, and many-to-many tagging.
- Implemented authentication (`login_required`), public/private recipe visibility with Django Q objects, open-redirect prevention, and PNG magic-byte upload validation.
- Developed a JSON search API with multi-field Q-object queries and prefix routing (`tag:`, `variation:`), plus a client-side guard that discards stale fetch responses to prevent race conditions.
- Built real-time JavaScript features: live ingredient aggregation, dynamic cook-time calculation, and instant search with DOM rebuilding.
- Designed a responsive UI with CSS Grid (4 breakpoints); deployed to AWS EC2 (Elastic IP) with Cloudflare DNS, GitHub Actions CI, and SFTP.

Simon Game | *C++, Qt Widgets* | *CS 3505*

Oct 2025

- Built a two-color Simon memory game with progressive difficulty (flash interval shrinks from 1000 ms to 200 ms).
- Added a Fun Mode that randomly repositions buttons before each flash, forcing players to track color rather than position.
- Decoupled model and UI with Qt signals and slots; sequenced animation timing with chained QTimer delays.

Online Multiplayer Snake Game | *C#, .NET 8, Blazor, MySQL, TCP Sockets* | *CS 3500*

Apr – May 2025

- Built a real-time multiplayer Snake game in three phases: a TCP networking library, a Blazor game client rendering HTML5 Canvas at ~50 FPS, and a MySQL-backed statistics server (81% of commits).

- Implemented a reusable TCP networking layer with thread-per-connection handling, a line-based message protocol, and thread-safe client tracking via fine-grained locks.
- Built the renderer: viewport camera transforms that follow the player's snake, per-player colors, wall-tile placement, and death and power-up animations.
- Wrote a standalone HTTP web server on raw HTTP (no ASP.NET) with regex-based URL routing to serve dynamic game and player statistics pages.

Spreadsheet Application | *C#, .NET 8, Blazor Server, System.Text.Json* | *CS 3500*

Feb – Apr 2025

- Developed a spreadsheet engine with an infix formula evaluator (dual-stack algorithm), a dependency graph with circular-reference detection and rollback, and JSON save/load (individual phase).
- Built the Blazor Server GUI with infinite scrolling via JavaScript interop, arrow-key navigation, and dynamic row/column expansion (group phase; 54% of commits).
- Wrote 143 MSTest unit tests covering the dependency graph, formula evaluator, and spreadsheet model, including a 200-element stress test with a 2-second timeout.

Web Development Projects | *HTML, CSS, JavaScript, Bootstrap* | *CS 1430*

May – Aug 2023

- Developed 13 front-end projects progressing from semantic HTML and CSS layout (Flexbox, Grid) to responsive Bootstrap sites and interactive JavaScript applications.
- Built a Tic-Tac-Toe engine with bitmask game state and win detection, a Mad Libs game using WordsAPI with regex-based verb conjugation, and a Wordle UI clone in CSS Grid.

Tower Defense Game | *Java, Swing, algs4* | *CS 2420*

Dec 2024

- Built a tower defense game in which tower placement reshapes the enemy route: removing a vertex's edges triggers BFS shortest-path recomputation, and DFS connectivity checks reject placements that would block the path (5-person team).
- Implemented a 60 FPS delta-time game loop with three tower types, three enemy types, 11 escalating waves, and Euclidean-distance targeting with linearly interpolated projectiles.

Technical Skills

Languages: C#, Python, C, C++, Java, JavaScript, SQL, HTML/CSS, LaTeX

Frameworks & Libraries: .NET, Blazor, Django, Qt 6, PyTorch, Hugging Face Transformers, Swing, Bootstrap

Game Development: Unreal Engine 5 (Blueprints, UMG), Unity 6, GameMaker Studio 2, Maya, ZBrush, Substance 3D Painter, Perforce

Systems & Networking: Linux, POSIX processes & signals, pthreads, TCP sockets, HTTP

Web & Infrastructure: Nginx, WordPress, AWS EC2, Cloudflare, MySQL, SQLite, GitHub Actions, Travis CI, Vagrant, VirtualBox

Tools: Git/GitHub, MSTest, Jupyter, Canva